

Audit Report

Own Protocol v2

Date: **June 24 – July 2, 2026**

Introduction

Own is a permissionless protocol for bringing tokenized real-world assets (RWAs) onchain. Users mint ERC-20 tokens called **eTokens** (e.g. eTSLA, eGOLD) using stablecoins; each eToken tracks the price of its underlying asset through onchain oracles and is backed by on-chain collateral held in Own Vaults. Where traditional RWA tokenization concentrates trust in a single custodian holding the physical asset, Own backs exposure with transparent, code-governed collateral vaults: any hedge fund, trading firm, or RWA custodian can permissionlessly operate a vault and back issued eTokens, with competition between them driving spreads.

Minting and redemption flow through a quote-driven RFQ marketplace: a buyer requests a firm off-chain quote from a market maker and either settles it atomically as a market order (`executeOrder`) or rests a limit order (`placeOrder`) for a maker to fill. Quotes are EIP-712 signed and authorized against a global signer registry, and settle inside a price band anchored to the protocol's oracle marks. Every holder also carries a code-enforced exit: a redeemer who cannot obtain a quote can force-execute against vault collateral at the oracle price after a claim window. Liquidity providers deposit collateral into ERC-4626 `OwnVaults` (with asynchronous deposit approval and queued withdrawals) and share the fees and yield the vault earns; a `BorrowManager` additionally lets users borrow stablecoins against their eToken collateral, with liquidity drawn from Aave v3 through each vault and standard accrual, liquidation, and bad-debt machinery.

The contracts under review comprise the full v2 core: `OwnMarket` (order placement, RFQ settlement, permissionless fills, force execution, and halted-asset redemption), `OwnVault` (LP collateral vaults), `VaultManager` (the global risk and control hub – exposure and collateral marks, utilization caps, the signer registry, and pause/halt controls), `AssetRegistry` (asset listing and token migration for corporate actions such as stock splits), `BorrowManager` (lending, liquidation, and bad-debt absorption), the in-house and Pyth oracle verifiers, the `EToken` with pass-through dividend distribution, and the WETH / wstETH / Aave periphery routers.

This report covers a security review of the Own Protocol v2 core contracts. The review combined full manual analysis of every core contract with multiple independent multi-agent automated audit passes, adversarial validation of externally generated findings, an arithmetic underflow/overflow sweep, and static analysis (Slither, 101 detectors – all raised alerts triaged, none valid). Every Critical and High finding was verified directly against source, and every fix carries a regression test that fails without it.

Scope

Repository: `own-protocol/own-v2`

Audited Commit: `9103c592083f35bd9aa57a349f4584386a140a05`

Files:

- `src/core/AssetRegistry.sol`
- `src/core/BorrowManager.sol`
- `src/core/OracleVerifier.sol`
- `src/core/OwnMarket.sol`
- `src/core/OwnVault.sol`
- `src/core/ProtocolRegistry.sol`
- `src/core/PythOracleVerifier.sol`
- `src/core/VaultManager.sol`
- `src/libraries/InterestRateModel.sol`
- `src/libraries/LendingMath.sol`
- `src/periphery/AaveRouter.sol`
- `src/periphery/WETHRouter.sol`
- `src/periphery/WstETHRouter.sol`
- `src/tokens/EToken.sol`
- `src/tokens/ETokenFactory.sol`

Findings

Each issue has an assigned severity:

- **Critical/High** issues are directly exploitable security vulnerabilities that can lead to loss of funds and need to be fixed.
- **Medium** issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- **Low/Info** issues are non-exploitable or informational findings that do not pose a direct security risk to the system's integrity.

Issues marked **RESOLVED** are fixed in the codebase and carry a regression test that fails without the fix. Issues marked **ACKNOWLEDGED** are accepted by design, with the rationale and any operational mitigations documented.

Issues Found

Critical	High	Medium	Low/Info
3	12	13	18

Issues Not Fixed and Not Acknowledged

Critical	High	Medium	Low/Info
0	0	0	0

Summary of Findings

ID	Severity	Title	Status
C-01	Critical	<code>verifyPrice</code> accepted any ever-signed price – stale signed prices could drain collateral	Fixed
C-02	Critical	Settlement price unbounded – a leaked quote-signer key could settle at an arbitrary price	Fixed
C-03	Critical	<code>claimThreshold</code> defaulted to zero – force execution exercisable the instant an order was placed	Fixed
H-01	High	Force-execution could drain a halted vault	Fixed
H-02	High	Resting redeem escrow stranded after <code>migrateToken</code> (stock split)	Fixed
H-03	High	Withdrawal gate read a stale collateral mark after a bad-debt release	Fixed
H-04	High	<code>absorbBadDebt</code> released collateral in 18-decimal units, not native decimals	Fixed
H-05	High	<code>fulfillWithdrawal</code> did not sync the collateral mark – utilization cap bypass	Fixed
H-06	High	Unbounded <code>releaseCollateral</code> drained pending-deposit escrow and bricked the vault	Fixed

H-07	High	No Aave health-factor gate on LP withdrawals or collateral release	Fixed
H-08	High	<code>liquidate</code> was callable on a halted asset at a divergent live price	Fixed
H-09	High	Unconditional Aave repay bricked every position-close path once the vault's pooled Aave debt hit zero	Fixed
H-10	High	Liquidation seize granularity stranded a dust crumb that permanently blocked <code>absorbBadDebt</code>	Fixed
H-11	High	<code>openExposure</code> ran the solvency cap against arbitrarily stale collateral marks	Fixed
H-12	High	<code>forceExecuteOrder</code> accepted any price proof within the order's lifetime window	Fixed
M-01	Medium	Pyth confidence interval ignored	Fixed
M-02	Medium	Unset oracle config accepted unbounded price pushes	Fixed
M-03	Medium	Unvalidated rate params could brick accrual – repay and liquidate DoS	Fixed
M-04	Medium	Withdrawal queue is not FIFO	Acknowledged
M-05	Medium	Book debt could lag real compounding Aave debt – LP shortfall	Fixed
M-06	Medium	Debt cap checked before <code>_accrue()</code> – cap bypass	Fixed
M-07	Medium	<code>borrow</code> ignored the bound vault's Paused/Halted status	Fixed
M-08	Medium	<code>_flooredIndex</code> exploded on a dust scaled-debt base; broke under multi-manager	Fixed
M-09	Medium	Sub-unit amounts recorded zero scaled debt while moving real value	Fixed
M-10	Medium	<code>WstETHRouter</code> wrapped the requested (not received) stETH – deposit DoS	Fixed
M-11	Medium	Redeem settle band bounded against a stale mark	Fixed
M-12	Medium	<code>migrateToken</code> on a halted asset desynced its frozen halt price	Fixed
M-13	Medium	<code>fulfillWithdrawal</code> can settle before a borrower's bad debt is absorbed	Acknowledged

L-01	Low	Protocol signatures used EIP-191 personal-sign instead of the documented EIP-712	Fixed
L-02	Low	Payment-token decimals and fee-on-transfer assumptions unenforced	Fixed
L-03	Low	<code>closeExposure</code> accepted a zero mark	Fixed
L-04	Low	Borrow manager is one-shot with no rotation	Acknowledged
L-05	Low	<code>migrateToken</code> accepted the current active or an existing legacy token	Fixed
L-06	Low	<code>sweepDividends</code> did not validate the token argument	Fixed
L-07	Low	<code>migrateToken</code> / <code>applySplit</code> were separately callable – migration atomicity	Fixed
L-08	Low	Oracle fee handling forwarded all of <code>msg.value</code>	Fixed
L-09	Low	Halt-settlement ceil rounding sweeps the surplus to the VaultManager	Acknowledged
L-10	Low	<code>settleHaltedPosition</code> tolerated a diverged payment token	Fixed
L-11	Low	<code>sweepDividends</code> pays <code>vault.manager()</code>	Acknowledged
L-12	Low	Fee-on-transfer tokens could under-fund order escrow	Fixed
L-13	Low	A stablecoin blacklist freeze could permanently brick escrow recovery	Fixed
L-14	Low	In-house <code>getPrice</code> had no read-time staleness bound	Fixed
L-15	Low	<code>maxDeposit</code> / <code>maxMint</code> misreported capacity to ERC-4626 integrators	Fixed
L-16	Low	<code>absorbBadDebt</code> over-charges LPs by a defaulted position's uncollected premium	Acknowledged
L-17	Low	<code>fillOrder</code> skipped the active-asset gate	Fixed
L-18	Low	<code>setMinAaveBorrowRateBps</code> unbounded – pathological floor could arithmetic-brick accrual	Fixed

Issue C-01: `verifyPrice` accepted any ever-signed price – stale signed prices could drain collateral

RESOLVED

Severity: **Critical**

Vulnerability Detail

The inline proof path `OracleVerifier.verifyPrice` (and the Pyth equivalent) checked only the signature and `price != 0` – never the timestamp – and the digest carries no nonce. Every price the signer ever produced therefore remained a valid proof forever. All consumers pass caller-supplied `priceData`, so an attacker could replay the most favorable historical price: over-release collateral in force-redeem (high asset + low collateral price), over-borrow against a stale high collateral price, or force-liquidate a healthy position with a stale low price.

Impact

Replay of any historical signed price enables direct collateral theft via force-redeem, borrow, or liquidation of healthy positions.

Resolution

Freshness is enforced in the consumers (`verifyPrice` stays a pure signature primitive):

`OwnMarket.forceExecuteOrder` settles at the user's own immutable `limitPrice` and requires the asset proof to be fresh ($\leq \text{priceMaxAge}$, not future-dated) and to currently satisfy the limit; `OwnMarket._convertToCollateral` and `BorrowManager._verifyPrice` reject proofs that are future-dated or older than `priceMaxAge`. `priceMaxAge` is governance-tunable (non-zero enforced); deployment value 2 minutes.

Issue C-02: Settlement price unbounded – a leaked quote-signer key could settle at an arbitrary price

RESOLVED

Severity: **Critical**

Vulnerability Detail

A leaked quote-signer key could settle `executeOrder` / `fillOrder` at an arbitrary price, draining the maker's full payment-token allowance in a single transaction. Nothing bounded the signed settlement price against the market's own view of the asset.

Impact

Full drain of the maker's payment-token allowance in one transaction under signer-key compromise.

Resolution

`OwnMarket._settleMint` / `_settleRedeem` now bound the settle price to $\pm \text{settleBandBps}$ of the `VaultManager` asset mark (revert `PriceOutOfBand`); deploy default 500 bps. Force-execute and `redeemHalted` are exempt (they settle at the limit / frozen halt price).

Issue C-03: `claimThreshold` defaulted to zero – force execution exercisable the instant an order was placed

RESOLVED

Severity: **Critical**

Vulnerability Detail

`claimThreshold` defaulted to 0 and no deployment script set it, making `forceExecuteOrder` exercisable immediately after order placement – the claim-delay protection the mechanism depends on simply did not exist in the default configuration.

Impact

The force-execution fallback could be used as an instant settlement path, bypassing the intended maker-response window.

Resolution

`forceExecuteOrder` reverts `ForceNotEnabled` while the threshold is 0; the setter rejects 0; the deploy and configuration scripts set 6 hours. All new risk parameters follow the same fail-safe-on-zero posture.

Issue H-01: Force-execution could drain a halted vault

RESOLVED

Severity: **High**

Vulnerability Detail

`forceExecuteOrder` accepted any registered vault as the collateral source. A halted vault's collateral is excluded from the global pool and reserved for its exiting LPs, but `releaseCollateral` had no status check – a redeemer could cherry-pick the halted vault and drain it.

Impact

Collateral reserved for a halted vault's exiting LPs could be drained by any redeemer.

Resolution

`forceExecuteOrder` reverts `VaultExcludedFromPool (vault)` when the VaultManager reports the vault as excluded.

Issue H-02: Resting redeem escrow stranded after `migrateToken` (stock split) RESOLVED

Severity: **High**

Vulnerability Detail

A resting redeem order escrowed the eToken resolved at placement, but every later path re-resolved the *current* active token. After a split migration the market held the old token while cancel / expire / fill / force targeted the new one – escrow recovery reverted, locking user funds. Borrow positions had the same staleness on their collateral token.

Impact

User escrow and borrow collateral permanently locked after a token migration.

Resolution

Migration is now a first-class convert-first flow: `Order.escrowToken` and `Position.collateralToken` snapshot the held token; cancel/expire return the snapshot; fill/force on a migrated order revert `OrderTokenMigrated`; `OwnMarket.convertLegacy` (pause/halt-exempt) converts legacy to active at the recorded ratio; `VaultManager.applySplit` re-denominates exposure USD-invariantly, so splits never move a position's health. `_settleMint` settles fills in `order.escrowToken`, not the current payment token.

Issue H-03: Withdrawal gate read a stale collateral mark after a bad-debt release

RESOLVED

Severity: **High**

Vulnerability Detail

`withdrawalBreachesUtil` nets against keeper-cached collateral marks.

`releaseCollateralForBadDebt` shrank real `totalAssets` without refreshing the mark, so in the window before the next keeper pull the gate under-reported utilization.

Impact

Informed LPs could exit ahead of a socialized loss.

Resolution

Mark updates are atomic with the release: `VaultManager.onCollateralReleased(amount)` (`onlyRegisteredVault`) decrements the mark proportionally and is called before the transfer in both collateral-out paths.

Issue H-04: `absorbBadDebt` released collateral in 18-decimal units, not native decimals

RESOLVED

Severity: **High**

Vulnerability Detail

The LP-socialized collateral slice was computed 18-decimal-normalized and passed straight to `releaseCollateralForBadDebt`, which transfers in native decimals. For 6-decimal collateral (aUSDC/USDC) the amount was 10^{12} too large – absorption reverted (bricked), or would over-release given sufficient balance. 18-decimal vaults were accidentally correct, which is why existing tests passed.

Impact

Bad-debt absorption bricked for 6-decimal collateral vaults, or massive over-release of collateral.

Resolution

A `BorrowManager._convertToCollateral` helper (mirroring `OwnMarket`'s) verifies the fresh collateral price and floors to native decimals; `absorbBadDebt` routes through it.

Issue H-05: `fulfillWithdrawal` did not sync the collateral mark – utilization cap bypass

RESOLVED

Severity: **High**

Vulnerability Detail

The third collateral-exit path missed the H-03 treatment: `fulfillWithdrawal` transferred collateral out without calling `onCollateralReleased`, leaving the cached mark stale-high. Serial withdrawals each cleared the gate against the un-decremented mark, and mints/borrows immediately after a withdrawal issued exposure/debt against collateral that had already left.

Impact

Global utilization cap bypass within a block; exposure and debt issued against departed collateral.

Resolution

`fulfillWithdrawal` (non-halted branch) calls `onCollateralReleased(assets)` after the gate check and before the transfer. Halted vaults skip it (already excluded from the pool).

Issue H-06: Unbounded `releaseCollateral` drained pending-deposit escrow and bricked the vault

RESOLVED

Severity: **High**

Vulnerability Detail

`releaseCollateral` (and `releaseCollateralForBadDebt`) transferred `amount` of `asset()` from the vault's raw balance with no `amount ≤ totalAssets()` bound. Since `totalAssets()` excludes the async approval-queue escrow, the raw balance exceeds `totalAssets()` whenever a deposit is pending. `forceExecuteOrder` let a redeemer pick any non-excluded vault and size the release with no per-vault free-collateral check, so a redeemer could force-execute against an approval-mode vault holding a pending deposit, pay out escrowed funds, then brick the vault – the next `totalAssets()` read underflow-reverts, freezing deposits, withdrawals, accept/reject/cancel, and keeper price pulls.

Impact

Pending-deposit escrow drained by a third party; vault permanently bricked via `totalAssets()` underflow.

Resolution

Both release paths revert `AmountExceedsBackedCollateral` when `amount > totalAssets()`, so a release can only spend share-backing collateral, never pending-deposit escrow – which also makes the underflow unreachable. The force-execute amplifier was additionally removed: collateral for a force execution is sourced only from an admin-vetted vault set, with use-time re-verification (registered / not excluded / non-RWA), and the H-07 health-factor gate bounds what any chosen vault can release.

Issue H-07: No Aave health-factor gate on LP withdrawals or collateral release

RESOLVED

Severity: **High**

Vulnerability Detail

`BorrowManager.borrow` draws stablecoin from Aave `onBehalf` of the vault, so the vault carries the Aave debt and its `aTokens` double as the Aave collateral backing it. But `totalAssets()` never nets that debt, and no withdrawal/release path checked the vault's Aave health factor. The sole withdrawal gate sees only synthetic exposure over *global* collateral; draining one vault barely moves the global total in a multi-vault pool, so the gate passes while a single vault's `aTokens` are pulled out from under its Aave borrow – tipping its Aave position into liquidation and socializing the penalty onto remaining LPs.

Impact

A vault's Aave position could be forced into liquidation, with the liquidation penalty socialized onto remaining LPs (first-mover advantage to early exiters).

Resolution

`BorrowManager.requireVaultHealthy()` (the same `getUserAccountData` check used by interest claims, against `minClaimHealthFactor = 1.1e18`) is called by `OwnVault.fulfillWithdrawal` (active vaults) and `releaseCollateral` after the `aTokens` leave, reverting `VaultUnsafeHealthFactor` if the release dropped the vault below the floor. Lending-disabled vaults and halted-vault emergency exits are exempt. Verified against live Aave V3 on Base via a fork test.

Issue H-08: `liquidate` was callable on a halted asset at a divergent live price

RESOLVED

Severity: **High**

Vulnerability Detail

The asset-level halt guard was wired into `borrow` and `migrateToken` but not into `liquidate`. A halted asset is meant to wind down only via the permissionless `settleHaltedPosition` at the single frozen halt price; leaving `liquidate` open created a second settlement path priced off a caller-supplied *live* signed proof. With the halt price above a still-fresh live price – the natural state at a halt – a liquidator could seize collateral cheaply at the live price, then redeem the seized eTokens at the higher frozen price through the permissionless `redeemHalted`.

Impact

Live-vs-halt price arbitrage drains borrower equity and the finite halt-redemption fund, and can manufacture residual bad debt.

Resolution

`liquidate` reverts `VaultEffectivelyHalted` when the asset is halted, before any price verification, so a halted position can only wind down through `settleHaltedPosition` at the frozen price. Asset *pause* is intentionally not gated – a paused asset has no frozen halt price and no `redeemHalted` path, so liquidation stays available as normal risk management.

Issue H-09: Unconditional Aave repay bricked every position-close path once the vault's pooled Aave debt hit zero

RESOLVED

Severity: **High**

Vulnerability Detail

All position-close paths – `repay`, `liquidate`, `absorbBadDebt`, `settleHaltedPosition` – route through `BorrowManager._repayAaveAndSweep`, which called `IAaveV3Pool.repay` unconditionally. Aave V3 reverts `NO_DEBT_OF_SELECTED_TYPE` when the borrower has no variable debt, and Aave lets anyone repay a position on another's behalf. An attacker could therefore – permissionlessly, for the cost of the debt – drive the vault's pooled Aave loan to zero while individual book positions were still open; every subsequent close then reverted, locking borrowers' eToken collateral with no rescue function. Permanent once the asset is halted or paused, since a fresh `borrow()` is blocked exactly when `settleHaltedPosition` is the only wind-down path.

Impact

All close paths bricked and borrower collateral locked – permanently for a halted or paused asset.

Resolution

`_repayAaveAndSweep` reads the vault's live Aave debt-token balance and skips the Aave call when it is zero, treating the whole amount as surplus (swept to the `VaultManager` – the existing zero-debt semantics). The test-suite Aave mock was made faithful (reverts on zero-debt repay) so this class cannot silently regress.

Issue H-10: Liquidation seize granularity stranded a dust crumb that permanently blocked `absorbBadDebt`

RESOLVED

Severity: **High**

Vulnerability Detail

`LendingMath.seizeAmount` floors and scales linearly, so a liquidation can only reduce a position's collateral in whole per-stable-unit seize steps. A liquidator winding down a deeply underwater position generically leaves a sub-step crumb $0 < r < \text{seize}(1).`_liquidate` then reverted `SeizeExceedsCollateral` for any further repay, and `absorbBadDebt` reverted `PositionStillCollateralized` because the crumb is non-zero – so the residual (meaningful) bad debt could never be cleared. The un-absorbable residual keeps a live LP-backed Aave slice and perturbs the interest index and utilization for every other borrower, with no admin remedy.$

Impact

Residual bad debt permanently unclearable – the generic outcome of any underwater liquidation.

Resolution

`_liquidate` caps the seize at the position's remaining collateral instead of reverting, so a deeply underwater position can always be driven to exactly zero collateral – the residual then becomes a zero-collateral position `absorbBadDebt` accepts. Capping only ever gives the liquidator *less* than the full bonus, so it is not an abuse vector.

Issue H-11: `openExposure` ran the solvency cap against arbitrarily stale collateral marks

RESOLVED

Severity: **High**

Vulnerability Detail

`openExposure` valued new exposure at the cached `assetMark` with no freshness check – the solvency cap could run against marks of any age, permitting exposure the current market would not.

Impact

New exposure could be opened against stale valuations, weakening the protocol's solvency guarantee.

Resolution

`openExposure` reverts `StaleAssetMark` if the asset mark is older than `maxMarkAge` (deploy default 15 minutes); `closeExposure` (risk-reducing) is exempt.

Issue H-12: `forceExecuteOrder` accepted any price proof within the order's lifetime window

RESOLVED

Severity: **High**

Vulnerability Detail

`forceExecuteOrder` accepted any asset proof timestamped within the order's `[createdAt, now]` window, so a stale favorable print – a wick the maker failed to execute against – could be exercised long after the market moved, costing LPs up to $(\text{limitPrice} - \text{currentPrice}) \times \text{remaining}$.

Impact

Stale favorable prices exercisable after the market moved, at LPs' expense.

Resolution

The asset leg now requires a fresh price ($\leq \text{priceMaxAge}$) that currently satisfies the limit (`StaleAssetPrice`), matching the collateral leg.

Issue M-01: Pyth confidence interval ignored

RESOLVED

Severity: **Medium**

Vulnerability Detail

`PythOracleVerifier._normalizePythPrice` read only `price/expo`, never `conf` – Pyth's own $\pm$ uncertainty band. Wide-uncertainty prices (thin market, degraded publishers) were consumed as exact by both the cached reads (marks) and inline proofs (force-execute, borrow, liquidate).

Impact

Pricing-sensitive paths consumed low-quality prices as exact.

Resolution

A `maxConfBps` bound (constructor param + admin setter, non-zero enforced) is checked in `_normalizePythPrice` – the chokepoint both read paths flow through (revert `ConfidenceTooWide`). Deploy default 100 bps. The liveness trade-off is intentional: while Pyth reports wider uncertainty than the bound, pricing for that asset pauses.

Issue M-02: Unset oracle config accepted unbounded price pushes

RESOLVED

Severity: **Medium**

Vulnerability Detail

`OracleVerifier.updatePrice` applied staleness/deviation bounds only when the per-asset config values were non-zero – an asset never configured via `setAssetOracleConfig` accepted any validly-signed push with no bounds. Exploitable without signer compromise: signed price blobs handed out for inline proofs share `updatePrice`'s digest format, so anyone could backfill a stale cache with the most favorable price ever signed.

Impact

Unbounded cache-price manipulation for any asset missing an off-chain configuration step.

Resolution

Fail closed: `updatePrice` reverts `OracleConfigNotSet` unless both bounds are configured, then applies them unconditionally; `setAssetOracleConfig` rejects zero values and emits a configuration event.

Issue M-03: Unvalidated rate params could brick accrual – repay and liquidate DoS RESOLVED

Severity: **Medium**

Vulnerability Detail

`InterestRateModel.premium` reverts when `optimalUtilBps == 0 || ≥ BPS`. Neither `setRateParams` nor the constructor validated the params, so a bad value made `_accrue` revert – bricking repay and liquidate, which must always stay open.

Impact

A misconfiguration could freeze all debt-closing operations.

Resolution

Both write sites reject invalid `optimalUtilBps` with `InvalidRateParams()`. Only `optimalUtilBps` is validated – it is the only value that can revert the curve; slope/base values can only misprice, not brick.

Issue M-04: Withdrawal queue is not FIFO

ACKNOWLEDGED

Severity: **Medium**

Vulnerability Detail

`fulfillWithdrawal` enforces only the wait period plus the utilization gate; near the cap, capacity is allocated by gas race / caller choice rather than request order.

Impact

Near the utilization cap, later requesters can exit before earlier ones.

Resolution

Acknowledged – the queue was never required to be FIFO; the FIFO claim was dropped from the specification (protocol docs and `Types.sol`). No code change.

Issue M-05: Book debt could lag real compounding Aave debt – LP shortfall

RESOLVED

Severity: **Medium**

Vulnerability Detail

`_accrue` sampled Aave's variable rate at sparse touch points and applied simple interest; Aave compounds continuously. During rate spikes book debt fell below the vault's real Aave debt, so borrower repayments under-covered the loan and LPs ate the shortfall.

Impact

Borrower repayments could under-cover the vault's real Aave loan during rate spikes.

Resolution

The interest index is floored so book debt never sits below the vault's real Aave debt, read live from the debt-token balance (`_flooredIndex`). Worst case the protocol premium shrinks toward zero during a spike, but borrowers always cover the Aave loan. A permissionless `accrue()` keeper entry point bounds the lag window.

Issue M-06: Debt cap checked before `_accrue()` – cap bypass

RESOLVED

Severity: **Medium**

Vulnerability Detail

`borrow` ran the hard cap against `totalDebtUSD()` before calling `_accrue()`, so the cap excluded interest accrued since the last touch – a borrow that should breach the cap could pass.

Impact

The global debt cap could be exceeded.

Resolution

`_accrue()` moved above the cap block in `borrow`, so the cap sees the accrued and floored debt.

Issue M-07: `borrow` ignored the bound vault's Paused/Halted status

RESOLVED

Severity: **Medium**

Vulnerability Detail

`_validateEligibility` checked the asset only, never the vault's status – a vault paused or halted during an incident kept being borrowed against via its Aave credit delegation.

Impact

New debt could be opened against a vault that governance had paused or halted.

Resolution

`_validateEligibility` reverts `VaultNotActive()` unless the vault is Active. Only `borrow` calls it, so repay / liquidate / halt settlement stay open on a non-Active vault – exits must always work.

Issue M-08: `_flooredIndex` exploded on a dust scaled-debt base; broke under multi-manager RESOLVED

Severity: **Medium**

Vulnerability Detail

The M-05 floor computed `minIndex = aaveDebt × 1e18 / _totalScaledDebt` – the vault's entire real Aave debt spread over this manager's recorded units. When repays wind the book down to a dust base while residual Aave debt (rounding crumbs, model lag) remains, the division explodes – e.g. a 50 USDC residual over 0.5 USDC of scaled units lifts the index $\times 101$, multiplying every remaining position's debt permanently (the index is monotonic). A second borrow manager on the same vault would likewise double-charge both books.

Impact

Permanent, uncorrectable over-charging of every remaining borrower.

Resolution

One borrow manager per vault is now a documented protocol invariant (already enforced on-chain – `setBorrowManager` is one-shot), and the floor is skipped when `_totalScaledDebt` is below one whole stablecoin unit: at dust scale there is no meaningful book left to protect, and residual Aave debt is crumb-scale by construction.

Issue M-09: Sub-unit amounts recorded zero scaled debt while moving real value

RESOLVED

Severity: **Medium**

Vulnerability Detail

Debt is stored as scaled units with floor division. Once the index exceeds 1.0, amounts below `index / 1e18` base units floor to zero scaled units with no guard, so real value moved while the ledger recorded nothing: a dust liquidation seized non-zero collateral (plus the bonus) while reducing debt by zero (repeatable griefing); a dust borrow recorded `principal = 0` – the 'no position' sentinel – leaving it unrepayable and its collateral stranded, with a second borrow silently orphaning the first; a dust repay took stablecoin and reduced nothing.

Impact

Repeatable griefing of borrower health and stranded collateral via dust-sized operations.

Resolution

Revert `AmountTooSmall()` whenever a non-zero input converts to zero scaled units, in `borrow`, `repay`, and `_liquidate`'s partial branch – the same guards Aave V3 carries. `settleHaltedPosition` is deliberately left ungated (wind-down must stay live; a zero there only under-reduces book debt, protocol-favorable).

Issue M-10: `WstETHRouter` wrapped the requested (not received) `stETH` – deposit DoS

RESOLVED

Severity: **Medium**

Vulnerability Detail

Lido's share math delivers 1-2 wei less than the `transferFrom` amount, so the router held less than `stETHAmount` and `wstETH.wrap(stETHAmount)` reverted on insufficient balance – breaking every `stETH` deposit.

Impact

All `stETH` deposits reverted.

Resolution

`_depositStETHInternal` measures the received balance-diff and wraps that; the existing `minSharesOut` check covers the wei-level shortfall.

Issue M-11: Redeem settle band bounded against a stale mark

RESOLVED

Severity: **Medium**

Vulnerability Detail

`_checkSettleBand` bounds the settle price to $\pm \text{settleBandBps}$ of the `VaultManager` mark (the C-02 leaked-key damage cap) but never checked the mark's age. The mint leg's subsequent `openExposure` enforces `maxMarkAge`, so a stale mark blocks mints; the redeem leg's `closeExposure` is freshness-exempt, so a leaked signer key – the exact threat the band exists for – could settle a redeem within band of a *stale* mark, draining the maker.

Impact

The signer-compromise damage cap was asymmetrically weaker on the redeem side.

Resolution

`_checkSettleBand` reverts `StaleSettleMark` when the mark is older than `maxMarkAge`, so both settle legs bound against a keeper-fresh mark. `closeExposure` stays intentionally stale-tolerant – it also serves `forceExecuteOrder` and `redeemHalted`, which must work when the mark is frozen.

Issue M-12: `migrateToken` on a halted asset desynced its frozen halt price

RESOLVED

Severity: **Medium**

Vulnerability Detail

Neither `AssetRegistry.migrateToken` nor `VaultManager.applySplit` guarded against a halted asset or rescaled the frozen halt price, while `applySplit` scales the global asset units. Splitting an already-halted asset doubles holders' token counts but leaves the halt price un-halved, so `redeemHalted` pays out at the un-rescaled price (2× at a 2:1 split), draining the finite halt-redemption fund.

Impact

Halt-redemption fund drained at a multiple of fair value after a split of a halted asset.

Resolution

`migrateToken` reverts `AssetHalted(ticker)` for a halted asset – a halted asset is in permanent wind-down and must never be re-denominated. The safe ordering (split first, then halt) is unaffected.

Issue M-13: `fulfillWithdrawal` can settle before a borrower's bad debt is absorbed

ACKNOWLEDGED

Severity: **Medium**

Vulnerability Detail

While a position is underwater but its loss is not yet booked into `totalAssets()` (via `absorbBadDebt`), an LP with a matured request exits at the pre-loss share price, socializing the loss onto the remaining LPs. Sibling of H-07 – whose Aave health-factor gate is blind to it, since a user default does not move the vault's own Aave health factor.

Impact

An early LP can socialize an unrecognized borrower loss onto remaining LPs.

Resolution

Acknowledged – there is no $O(1)$ on-chain signal for unrecognized bad debt (the loss window opens at 'position underwater', before liquidation). Mitigated operationally by a proactive pause: `OwnVault.pause()` freezes deposits and withdrawals when a backing asset turns volatile, before the window opens, with an over-biased automated trigger. To be revisited with a price-aware withdrawal gate if it becomes material.

Low Severity Findings

L-01: Protocol signatures used EIP-191 personal-sign instead of the documented EIP-712 RESOLVED

All protocol signatures migrated to EIP-712 typed data: quotes sign a structured `Quote` and price attestations a `PriceAttestation`, both under the domain `("Own Protocol", "1", chainId, verifyingContract)`. Foreign-domain replay tests (wrong chainId / verifying contract) lock the encoding.

L-02: Payment-token decimals and fee-on-transfer assumptions unenforced RESOLVED

Covered by existing code: `setPaymentToken` enforces `decimals() ≤ 18`, and the fee-on-transfer half is closed by the L-12 escrow balance-diff check.

L-03: `closeExposure` accepted a zero mark RESOLVED

Now reverts `PriceUnavailable` on a zero mark, mirroring `openExposure` (reachable via an extreme `applySplit` ratio flooring the mark).

L-04: Borrow manager is one-shot with no rotation ACKNOWLEDGED

By design – one borrow manager per vault for the vault's lifetime is the documented invariant the M-08 fix relies on; rotation would break the interest-index floor's debt attribution. The incident path for a compromised manager is `haltVault` (instant LP withdrawals, collateral excluded from the pool).

L-05: `migrateToken` accepted the current active or an existing legacy token RESOLVED

Now rejects both with `InvalidNewToken`.

L-06: `sweepDividends` did not validate the token argument RESOLVED

Now validated via `AssetRegistry.isValidToken(InvalidEToken)`; legacy tokens stay sweepable.

L-07: `migrateToken` / `applySplit` were separately callable – migration atomicity RESOLVED

`AssetRegistry.migrateToken` now calls `VaultManager.applySplit` atomically in the same transaction with a single ratio, and `applySplit` is locked to the `AssetRegistry`, so it can be neither called nor skipped independently.

L-08: Oracle fee handling forwarded all of `msg.value`

RESOLVED

`_verifyPrice` forwards exactly `verifyFee(priceData)`, and `borrow / liquidate / absorbBadDebt` refund the surplus to the caller as their last step. Also fixes a latent double-forward in `absorbBadDebt` (two price proofs, each previously sent the full `msg.value`).

L-09: Halt-settlement ceil rounding sweeps the surplus to the VaultManager

ACKNOWLEDGED

Accepted – the surplus is bounded by one rounding step (sub-cent) per settlement; a refund would cost more in gas and add a freeze-revert surface for negligible value. Documented in code.

L-10: `settleHaltedPosition` tolerated a diverged payment token

RESOLVED

Now requires `vaultManager.paymentToken() == stablecoin(PaymentTokenMismatch)` – a diverged payment token fails loud and recoverable instead of mis-accounting.

L-11: `sweepDividends` pays `vault.manager()`

ACKNOWLEDGED

By design – the revenue model routes collateral dividends to the VaultManager, the same destination as the lending-premium sweep; the admin-mutable `setManager` is the same trust boundary that governs all other VM-directed flows.

L-12: Fee-on-transfer tokens could under-fund order escrow

RESOLVED

`placeOrder` measures the escrow balance-diff and reverts `FeeOnTransferNotSupported` when `received != sent`. Direct party-to-party legs are intentionally not gated – a fee-on-transfer token there shorts the counterparty, not the escrow pool.

L-13: A stablecoin blacklist freeze could permanently brick escrow recovery

RESOLVED

`_returnEscrow` routes through `_pushOrSweep` – if the push to the user fails, the escrow sweeps to the governance treasury and emits `EscrowSweptToTreasury`; resolution is off-chain. The halt-funding half is operational: `setHaltRedeemAddress` is rotatable and the docs mandate monitoring plus immediate rotation on freeze.

L-14: In-house `getPrice` had no read-time staleness bound

RESOLVED

`OracleVerifier.getPrice` now reverts `StalePrice` when the cached price is older than the asset's configured `maxStaleness` (mirroring the Pyth verifier), so a keeper can no longer re-stamp a stale in-house price as fresh via `pullAssetPrice`.

L-15: `maxDeposit` / `maxMint` misreported capacity to ERC-4626 integrators

RESOLVED

Both returned `type(uint256).max` while `deposit/mint` would revert for the caller (approval gate / `onlyManager`). Both now return 0 for callers who cannot actually deposit or mint, keyed on `msg.sender`.

L-16: `absorbBadDebt` over-charges LPs by a defaulted position's uncollected premium

ACKNOWLEDGED

With the natural `absorbAmount = 0`, LPs eat the real Aave hole plus the premium the VaultManager also collects – over-socializing by the position's uncollected premium. Accepted: the path is operator-only, bounded by per-position premium, and the operator neutralizes it by setting `absorbAmount` to the position's earned interest.

L-17: `fillOrder` skipped the active-asset gate

RESOLVED

A resting Mint limit order could be filled – opening new exposure and minting eTokens – after an operator deactivated the asset. `fillOrder` now validates the asset on the Mint branch only (Redeem fills stay exempt so existing positions wind down). All risk caps still bounded the impact – a control-surface gap, not a solvency breach.

L-18: `setMinAaveBorrowRateBps` unbounded – pathological floor could arithmetic-brick accrual

RESOLVED

The floor feeds a raw `rate × dt` multiply, so a pathological `uint256` value overflows and reverts `_accrue()`, bricking every borrow operation until reset (admin-only, recoverable). Now reverts `InvalidMinAaveBorrowRate` above 100% APR.